



Zoo $\leftrightarrow$ Hanzo

Settlement Integration: AI Compute Payments from Zoo Applications

Antje Worring & Zach Kelling

Zoo Labs Foundation

March 2025

Abstract

We present the settlement integration between Zoo DeFi protocols and the Hanzo AI compute marketplace. The protocol enables \$ZOO token holders to pay for AI inference, model training, and agent execution on Hanzo infrastructure, with final settlement on Lux C-Chain. It introduces a three-phase flow: (1) pre-authorization on Zoo L2, where \$ZOO tokens are escrowed in a Settlement Vault with configurable payment modes (prepaid, streaming, postpaid); (2) compute execution on Hanzo, where the Compute Oracle attests to usage via signed reports; and (3) finalization on Lux C-Chain, where the Settlement Ledger records the immutable payment record with provider share, conservation share, and protocol fee. Conservation impact is tracked end-to-end: a configurable fraction of compute spending (default 2%) generates conservation credits per ZIP-0700. The protocol supports DeFi composability via compute staking, streaming payments, and inference pools.

Keywords: settlement, AI compute, micropayments, streaming, Hanzo, conservation impact

1 Introduction

The Zoo, Hanzo, and Lux ecosystems form a complementary triad:

- **Zoo:** Community, governance, conservation funding, and DeFi liquidity.
- **Hanzo:** AI compute infrastructure, LLM gateway (100+ providers), and agent frameworks.
- **Lux:** Settlement finality, cross-chain security, and DeFi infrastructure.

Zoo applications—avatar tutors for conservation education, habitat modeling with satellite imagery, species classification from camera traps—require AI compute from Hanzo. Today, this compute is paid via traditional payment rails, disconnected from the on-chain economy. ZIP-0801 unifies settlement so that \$ZOO tokens are a first-class payment method for AI compute, settlement occurs on Lux for finality and auditability, and conservation impact flows automatically from every inference call.

1.1 Contributions

1. The ISettlementVault interface for escrowed compute payments on Zoo L2 (Section 3).
2. The ISettlementLedger interface for final records on Lux C-Chain (Section 4).
3. The IComputeOracle interface for attested usage reporting (Section 5).
4. The three-phase settlement flow with conservation impact tracking (Section 6).
5. DeFi composability via compute staking, streaming, and inference pools (Section 8).
6. Security analysis of oracle compromise, escrow drain, and price manipulation (Section 10).

2 Architecture

3 Settlement Vault

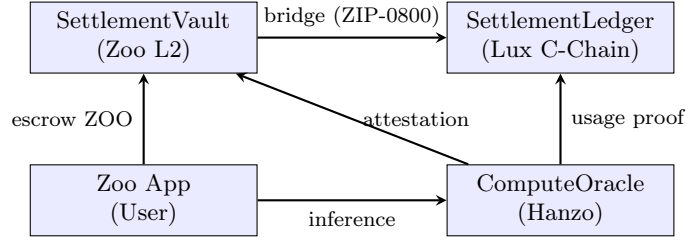


Figure 1: Zoo ↔ Hanzo settlement architecture.

```

1 interface ISettlementVault {
2     enum PaymentMode {
3         PREPAID,    // Full amount escrowed upfront
4         STREAMING,  // Per-second streaming payment
5         POSTPAID    // Credit-based, settled after
6     }
7
8     struct ComputeSession {
9         bytes32 sessionId;
10        address payer;
11        address hanzoEndpoint;
12        PaymentMode mode;
13        uint256 maxAmount;
14        uint256 consumedAmount;
15        uint256 ratePerSecond; // For STREAMING
16        uint256 startTimestamp;
17        uint256 endTimestamp;  // 0 = active
18        uint16 conservationBps;
19        bool finalized;
20    }
21
22    function createSession(
23        address hanzoEndpoint,
24        PaymentMode mode,
25        uint256 maxAmount,
26        uint256 ratePerSecond,
27        uint16 conservationBps
28    ) external returns (bytes32 sessionId);
29
30    function recordUsage(
31        bytes32 sessionId,
32        uint256 amount,
33        bytes calldata oracleAttestation
34    ) external;
35
36    function finalizeSession(
37        bytes32 sessionId
38    ) external;
39
40    function cancelSession(
41        bytes32 sessionId
42    ) external;
43
44    event SessionCreated(
45        bytes32 indexed sessionId,
46        address indexed payer,

```

```

47         address indexed hanzoEndpoint,
48         PaymentMode mode,
49         uint256 maxAmount
50     );
51     event UsageRecorded(
52         bytes32 indexed sessionId,
53         uint256 amount,
54         uint256 totalConsumed
55     );
56     event SessionFinalized(
57         bytes32 indexed sessionId,
58         uint256 totalPaid,
59         uint256 conservationDonation,
60         uint256 refunded
61     );
62 }

```

Listing 1: ISettlementVault interface

4 Settlement Ledger

```

1  interface ISettlementLedger {
2      struct SettlementRecord {
3          bytes32 sessionId;
4          address payer;
5          address provider;
6          uint256 totalAmount;
7          uint256 providerShare;
8          uint256 conservationShare;
9          uint256 protocolFee;
10         uint256 timestamp;
11         bytes32 usageProofHash;
12     }
13
14     function processSettlement(
15         bytes32 sessionId,
16         address payer,
17         address provider,
18         uint256 totalAmount,
19         uint256 conservationBps,
20         bytes32 usageProofHash
21     ) external;
22
23     function getSettlement(bytes32 sessionId)
24         external view returns (
25             SettlementRecord memory);
26     function providerTotalSettled(
27         address provider
28     ) external view returns (uint256);
29     function totalConservationImpact()
30         external view returns (uint256);
31
32     event SettlementProcessed(
33         bytes32 indexed sessionId,
34         address indexed payer,
35         address indexed provider,
36         uint256 totalAmount,

```

```

37         uint256 conservationShare
38     );
39 }

```

Listing 2: ISettlementLedger interface

5 Compute Oracle

```

1  interface IComputeOracle {
2      struct UsageReport {
3          bytes32 sessionId;
4          uint256 tokensProcessed; // LLM tokens
5          uint256 computeSeconds; // GPU-seconds
6          uint256 zooAmount;      // ZOO for usage
7          string model;           // e.g., "zen-72b"
8          uint256 timestamp;
9          bytes signature;        // Operator sig
10     }
11
12     function submitReport(
13         UsageReport calldata report
14     ) external;
15     function operator()
16         external view returns (address);
17     function pricePerMillionTokens(
18         string calldata model
19     ) external view returns (uint256);
20
21     event UsageReported(
22         bytes32 indexed sessionId,
23         uint256 tokensProcessed,
24         uint256 zooAmount,
25         string model
26     );
27 }

```

Listing 3: IComputeOracle interface

The oracle is the minimal trusted component: it attests to off-chain compute usage. Its operator is registered in the Zoo Contract Registry (ZIP-0100) and can be rotated by governance.

6 Settlement Flow

6.1 Phase 1: Pre-Authorization (Zoo L2)

1. User calls `createSession()` with payment mode and maximum \$ZOO.
2. \$ZOO tokens transferred from user to vault (escrowed).
3. `sessionId` returned and communicated to Hanzo via API.

6.2 Phase 2: Compute Execution (Hanzo)

1. User sends inference/training requests to Hanzo API with `sessionId` in headers.

2. Hanzo executes the workload and meters usage.
3. Compute Oracle periodically submits **UsageReport** to the Settlement Vault.
4. For STREAMING mode, the vault continuously drains escrow at **ratePerSecond**.

```

1  POST /v1/chat/completions HTTP/1.1
2  Host: llm.hanzo.ai
3  Authorization: Bearer <hanzo-api-key>
4  X-Zoo-Session-Id: 0xabc123...
5  X-Zoo-Payer: 0x742d35Cc...
6  Content-Type: application/json
7
8  {
9    "model": "zen-72b",
10   "messages": [
11     {"role": "user",
12      "content": "Analyze satellite imagery
13               for deforestation"}
14   ],
15   "max_tokens": 4096
16 }

```

Listing 4: Hanzo API integration via HTTP headers

6.3 Phase 3: Finalization (Lux C-Chain)

1. Session ends (user-initiated, timeout, or escrow depleted).
2. `finalizeSession()` computes the split:
 - Provider share: $\text{consumed} * (10000 - \text{conservBps} - \text{protocolBps}) / 10000$
 - Conservation share: $\text{consumed} * \text{conservBps} / 10000$
 - Protocol fee: $\text{consumed} * \text{protocolBps} / 10000$ (default 100 bps = 1%)
 - Refund: `maxAmount - consumed` returned to payer
3. Provider share and protocol fee bridged to Lux C-Chain via ZIP-0800.
4. Conservation share sent to Zoo conservation fund on Zoo L2.
5. Settlement Ledger records the final settlement with `usageProofHash`.

7 Pricing

Prices are updated daily based on the ZOO/USD TWAP from Lux DEX pools. The oracle publishes prices on-chain and signs each update.

Model Tier	ZOO / 1M Input	ZOO / 1M Output
Small (< 10B params)	50 ZOO	150 ZOO
Medium (10B–100B)	200 ZOO	600 ZOO
Large (100B–500B)	500 ZOO	1,500 ZOO
Frontier (500B+)	2,000 ZOO	6,000 ZOO

Table 1: Compute pricing by model tier (ZOO per 1M tokens).

8 DeFi Composability

8.1 Compute Staking

Stake \$ZOO to earn compute credits at a discount. Staked \$ZOO generates conservation impact credits passively:

$$\text{credits_per_year} = S_{\text{staked}} \cdot r_{\text{conservation}} \quad (1)$$

where $r_{\text{conservation}} = 0.02$ (2% annual conservation yield on staked compute).

8.2 Streaming Payments

Pay-per-second for long-running training jobs:

$$\text{amount_consumed}(t) = r \cdot (t - t_0) \quad (2)$$

where r is the rate in ZOO/second and t_0 is the session start. The vault halts streaming when escrow is depleted.

8.3 Inference Pools

Liquidity pools where \$ZOO providers fund shared compute access:

$$\text{LP_yield} = \frac{\text{session_fees} \cdot (1 - \text{conservation_bps}/10000)}{\text{total_pool_liquidity}} \quad (3)$$

LPs earn a share of settlement fees proportional to their contribution.

8.4 Compute Futures

Pre-purchase compute at today’s prices for future use. Backed by \$ZOO escrow in the Settlement Vault. The futures contract locks in the current `pricePerMillionTokens` for a specified duration.

9 Conservation Impact

Every compute session generates conservation impact:

1. The `conservationBps` parameter (default 200 bps = 2%) determines the fraction directed to conservation.
2. Conservation \$ZOO sent to the default conservation fund (ZIP-0100).

3. The payer receives non-transferable impact credits (per ZIP-0700).
4. The Settlement Ledger maintains a running total queryable by anyone.

Theorem 9.1 (Conservation from Compute). *If Zoo applications generate V ZOO in annual compute spending with conservation rate c , the annual conservation funding from compute alone is:*

$$F_{\text{conservation}} = V \cdot \frac{c}{10000} \quad (4)$$

With $V = 10,000,000$ ZOO/year and $c = 200$ bps, $F_{\text{conservation}} = 200,000$ ZOO/year directed to conservation from AI inference alone.

10 Security Analysis

10.1 Oracle Compromise

A compromised Compute Oracle could report false usage and drain escrowed \$ZOO. Mitigations:

- Per-session spending caps (`maxAmount`).
- Anomaly detection on usage patterns.
- Governance ability to pause and replace the oracle.

10.2 Escrow Drain

STREAMING mode continuously drains escrow. Users MUST set appropriate `maxAmount` limits. The vault MUST halt streaming when escrow is depleted.

10.3 Price Manipulation

ZOO/USD price feeds could be manipulated via flash loans on Lux DEXs. The oracle uses 24-hour TWAP, not spot price.

10.4 Bridge Latency

Settlement finalization requires a bridge message from Zoo to Lux. During bridge congestion, finalization may be delayed. The vault holds escrowed funds safely until finalization completes.

10.5 Replay Protection

Each `UsageReport` includes a `sessionId` and `timestamp`. The vault tracks cumulative usage per session and rejects reports that would exceed `maxAmount`.

10.6 Provider Rug

A provider could accept payment and not deliver compute. Usage reports are signed by the Compute Oracle, not the provider. The oracle independently verifies compute execution before attesting.

11 Related Work

Superfluid [1] provides streaming payment primitives; the Settlement Vault’s STREAMING mode uses analogous constant-flow mechanics. Filecoin [2] settles storage payments on-chain with proof-of-replication; our Compute Oracle provides analogous attestation for AI compute. Ritual [3] integrates AI inference with blockchain settlement; our three-phase architecture provides additional separation of concerns. Akash Network [4] provides a decentralized compute marketplace; our protocol specifically integrates with the Hanzo centralized compute gateway while maintaining on-chain settlement for auditability.

The conservation impact layer is unique to the Zoo ecosystem: no existing compute payment protocol directs a fraction of spending to ecological outcomes.

12 Conclusion

The Zoo $\leftrightarrow$ Hanzo settlement integration makes \$ZOO a first-class payment method for AI compute. The three-phase architecture—pre-authorization on Zoo L2, execution on Hanzo, finalization on Lux C-Chain—separates concerns across the stack while maintaining end-to-end auditability. Conservation impact from every inference call creates a sustainable funding mechanism where AI usage directly supports ecological outcomes.

Acknowledgments

This work was supported by Zoo Labs Foundation. We thank the Hanzo AI team for the compute gateway integration and the Lux team for settlement layer support.

References

- [1] Superfluid, “Superfluid Protocol: Real-Time Finance,” <https://docs.superfluid.finance>, 2021.
- [2] Protocol Labs, “Filecoin: A Decentralized Storage Network,” Technical Report, 2017.
- [3] Ritual, “Ritual: Sovereign Execution Layer for AI,” Technical Documentation, 2023.
- [4] Akash Network, “Akash: The Decentralized Cloud,” Technical Documentation, 2020.
- [5] Hanzo AI, “HIP-0101: Hanzo-Lux Bridge Protocol,” Hanzo Improvement Proposals, 2023.
- [6] Hanzo AI, “HIP-0044: API Gateway,” Hanzo Improvement Proposals, 2023.
- [7] Hanzo AI, “HIP-0040: Multi-Language SDK,” Hanzo Improvement Proposals, 2023.
- [8] Zoo Labs Foundation, “ZIP-0015: Zoo L2 Chain Architecture,” Zoo Improvement Proposals, 2024.
- [9] Zoo Labs Foundation, “ZIP-0042: Cross-Ecosystem Interoperability,” Zoo Improvement Proposals, 2024.
- [10] Zoo Labs Foundation, “ZIP-0100: Zoo Contract Registry,” Zoo Improvement Proposals, 2024.

- [11] Zoo Labs Foundation, “ZIP-0500: ESG Principles for Conservation Impact,” Zoo Improvement Proposals, 2024.
- [12] Zoo Labs Foundation, “ZIP-0700: ZRC-20 Fungible Token Standard,” Zoo Improvement Proposals, 2025.
- [13] Zoo Labs Foundation, “ZIP-0800: Zoo-Lux Bridge Protocol,” Zoo Improvement Proposals, 2025.
- [14] F. Vogelsteller and V. Buterin, “EIP-20: Token Standard,” Ethereum Improvement Proposals, 2015.
- [15] S. Nazarov et al., “Chainlink 2.0: Next Steps in Decentralized Oracle Networks,” Technical Report, 2021.
- [16] H. Adams et al., “Uniswap v3 Core,” Technical Documentation, 2021.
- [17] Aave, “Aave Protocol V2,” Technical Documentation, 2020.
- [18] A. Martinez and M. San Juan, “EIP-3156: Flash Loans,” Ethereum Improvement Proposals, 2020.
- [19] OpenZeppelin, “OpenZeppelin Contracts,” <https://openzeppelin.com/contracts>, 2023.
- [20] Gelato Network, “Gelato: Automate Smart Contract Executions,” Technical Documentation, 2022.